STUDY BOT - AI STUDY ASSISTANT

Project Report

Submitted by: Syeda Alia Samia

Date: February 23, 2026

Project prepared for:

DevTown / Codetown Edutech Pvt. Ltd.

1. PROJECT OVERVIEW

Study Bot is an AI-powered chatbot that helps students with study-related questions and remembers previous conversations using MongoDB. The bot provides contextual responses by storing chat history and retrieving it for each interaction.

Technologies Used:

- FastAPI (Python web framework)
- Groq LLM with Llama-3.3-70b-versatile model
- MongoDB Atlas (cloud database)
- LangChain for AI integration
- Render for cloud deployment
- PyMongo as MongoDB driver (version 4.11.3)

Key Features:

- Answer academic questions on various topics
- Maintain conversation context across sessions
- Store complete chat history in MongoDB
- RESTful API endpoints for easy integration
- Deployed and accessible 24/7 on Render

---

2. MEMORY IMPLEMENTATION (MongoDB)

The bot uses MongoDB Atlas to persist conversation history, enabling context-aware responses.

Database Schema:

Collection: chat_history

```
{
"session_id": "unique-uuid-string",
"user_id": "student@example.com",
"messages": [
{
"role": "user",
"content": "What is photosynthesis?",
"timestamp": "2026-02-23T10:30:00Z"
```

},
{
"role": "assistant",
"content": "Photosynthesis is the process...",
"timestamp": "2026-02-23T10:30:05Z"
}
],
"created_at": "2026-02-23T10:30:00Z",
"updated_at": "2026-02-23T10:30:05Z"
}

How Memory Works:

1. When a user sends a message, the bot checks for an existing session_id
2. If no session_id is provided, a new UUID is generated
3. The last 10 messages from that session are retrieved from MongoDB
4. These messages are formatted and sent to the LLM along with the new question
5. The AI generates a response with full conversation context
6. Both the user message and bot response are saved to MongoDB
7. Indexes on session_id and timestamp ensure fast retrieval

Code Implementation (database.py):

```
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure

class MongoDB:
client = None
database = None

text

@classmethod

def connect_to_mongodb(cls):

    cls.client = MongoClient(settings.MONGODB_URI)

    cls.database = cls.client[settings.DATABASE_NAME]

    cls.database[settings.COLLECTION_NAME].create_index("session_id")

    cls.database[settings.COLLECTION_NAME].create_index("timestamp")
```

```
@classmethod
def close_connection(cls):
    if cls.client:
        cls.client.close()
```

---

3. API DOCUMENTATION

Base URL: https://study-bot-aliasyeda-duva.onrender.com

Endpoint 1: Chat

POST /api/chat
Content-Type: application/json

Request Body:
{
"user_id": "student123",
"message": "What is quantum physics?",
"session_id": null
}

Response:
{
"session_id": "abc123-def456",
"user_id": "student123",
"message": "What is quantum physics?",
"response": "Quantum physics is the study of matter and energy at the atomic level...",
"timestamp": "2026-02-23T10:30:05Z"
}

Endpoint 2: Get Chat History

GET /api/chat/history/{session_id}

Response:
{
"session_id": "abc123-def456",
"history": [
{
"role": "user",
"content": "What is quantum physics?",
"timestamp": "2026-02-23T10:30:00Z"
},

{
"role": "assistant",
"content": "Quantum physics is the study...",
"timestamp": "2026-02-23T10:30:05Z"
}
]
}

Endpoint 3: Delete Session

DELETE /api/chat/session/{session_id}

Response:
{
"message": "Session abc123-def456 deleted"
}

Endpoint 4: Health Check

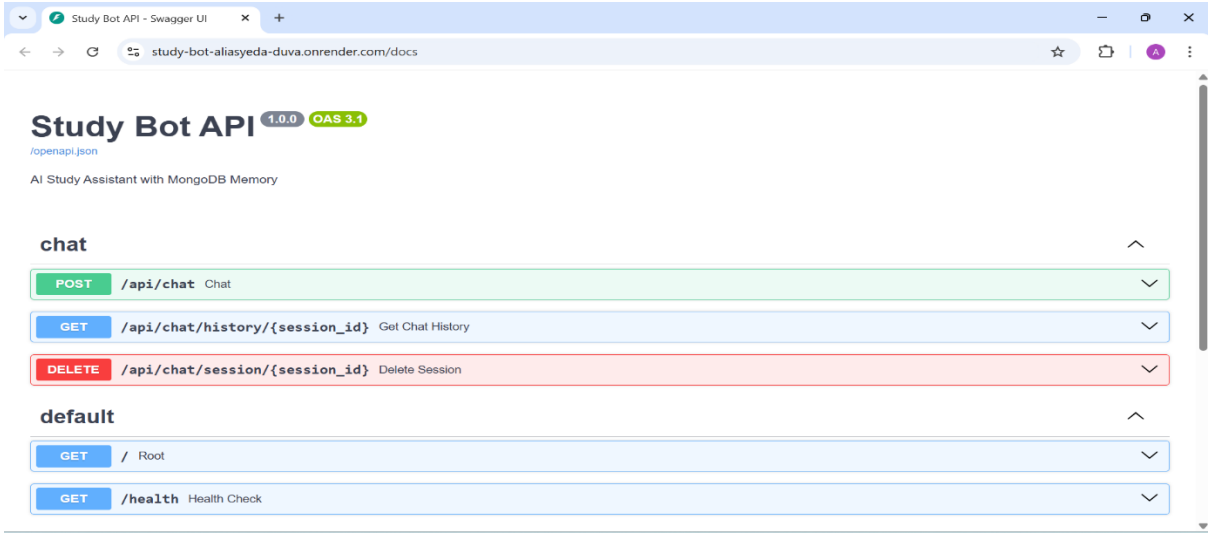
GET /health

Response:
{
"status": "healthy",
"database": "connected"
}

4. TEST SCREENSHOTS

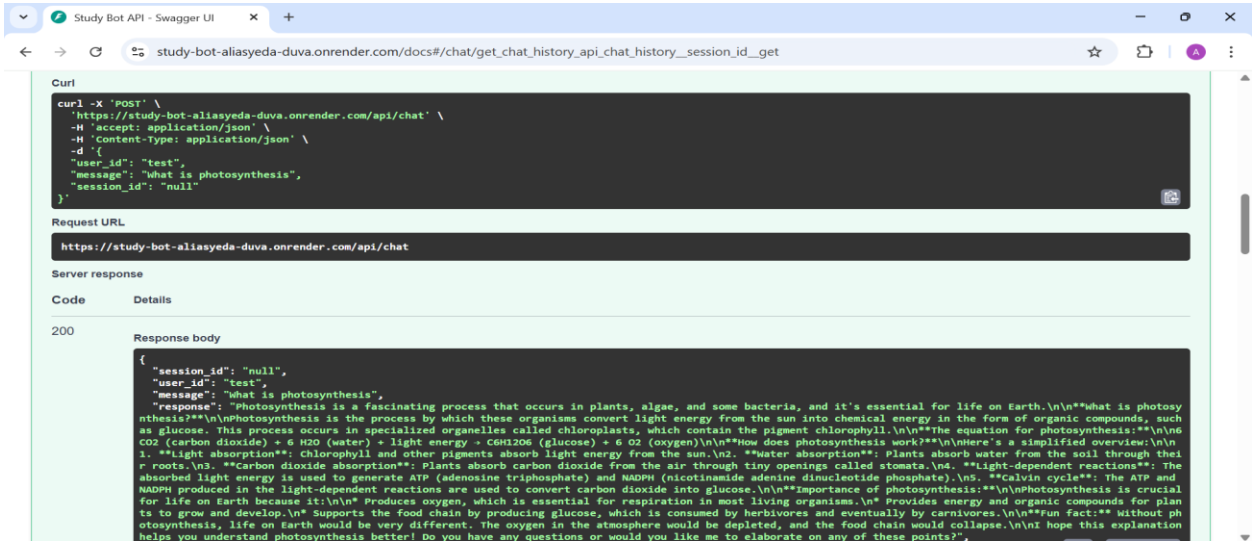
SCREENSHOT 1: SWAGGER UI

FastAPI Swagger documentation showing all available API endpoints



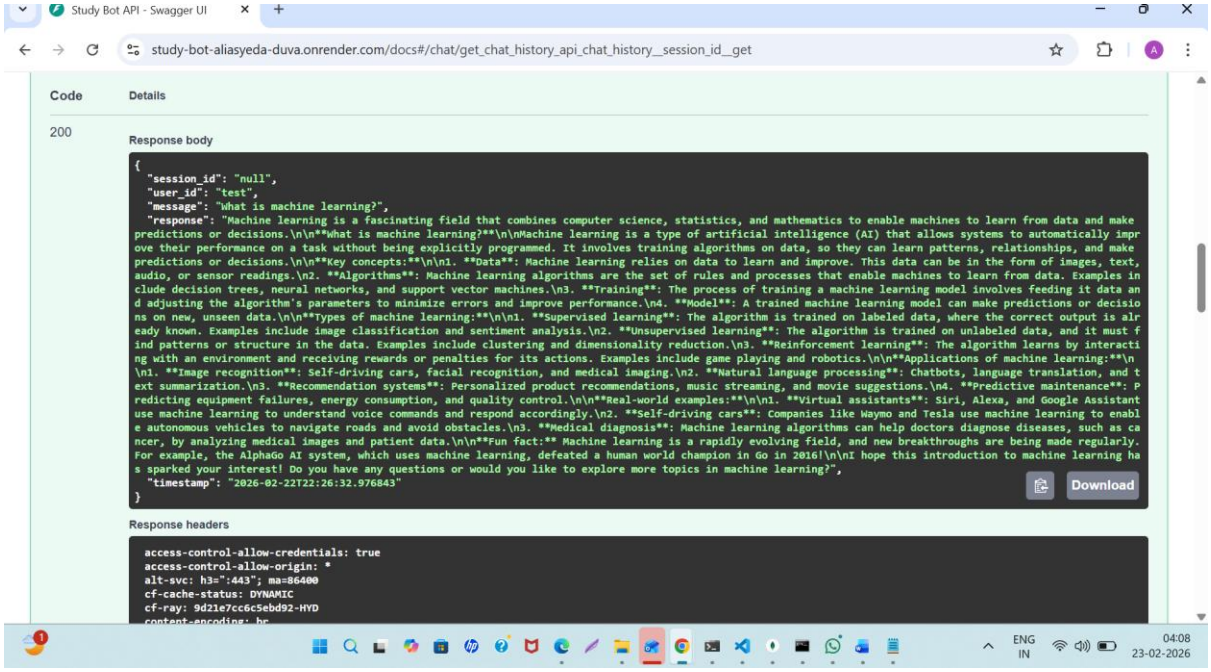
SCREENSHOT 2: FIRST CHAT INTERACTION]

screenshot here showing:
Request: {"user_id": "test", "message": "What is photosynthesis?"}
Response: Bot's detailed explanation
Caption: Study Bot responding to a biology question

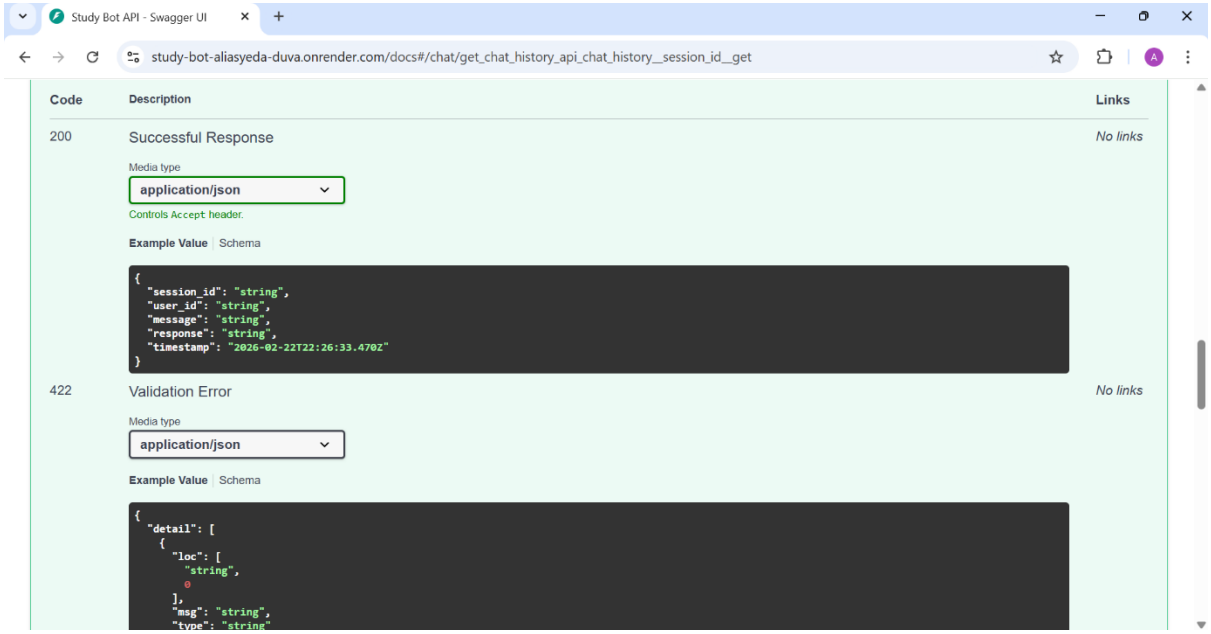


SCREENSHOT 3: CONTEXT-AWARE RESPONSE

screenshot here showing:
First message: "Explain machine learning"
Response referencing machine learning
Caption: Bot remembers previous conversation and provides contextual follow-up



More Screenshots:



Study Bot API - Swagger UI

study-bot-aliasyeda-duva.onrender.com/docs#/chat/get_chat_history_api_chat_history__session_id__get

```
    }
  ]
}
```

GET `/api/chat/history/{session_id}` Get Chat History

Endpoint: GET /api/chat/history/{session_id} What it does: Returns all messages from a conversation

Parameters | Try it out

Name	Description
session_id * required string (path)	session_id

Responses

Code	Description	Links
200	Successful Response	No links

Study Bot API - Swagger UI

study-bot-aliasyeda-duva.onrender.com/docs#/chat/get_chat_history_api_chat_history__session_id__get

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema `"string"`	No links
422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

Study Bot API - Swagger UI

study-bot-aliasyeda-duva.onrender.com/docs#/chat/get_chat_history_api_chat_history__session_id__get

DELETE `/api/chat/session/{session_id}` Delete Session

default

GET / Root

GET `/health` Health Check

Schemas

- ChatRequest > Expand all object
- ChatResponse > Expand all object
- HTTPValidationError > Expand all object
- ValidationError > Expand all object

5. DEPLOYMENT DETAILS

Live API URL:
https://study-bot-aliasyeda-duva.onrender.com

GitHub Repository:
https://github.com/aliasyeda/Study-bot

Deployment Platform: Render.com (Free Tier)

Environment Variables Configured:

GROQ_API_KEY = [configured in Render dashboard]
MONGODB_URI =
mongodb+srv://studybotuser:password@cluster0.hddcfbu.mongodb.net/study_bot
DATABASE_NAME = study_bot
COLLECTION_NAME = chat_history

Deployment Challenges Solved:

1. Package compatibility with Python 3.14
2. MongoDB connection SSL handshake issues
3. IP whitelisting for Render's outbound addresses
4. Authentication configuration
5. Port binding configuration

6. SAMPLE TEST COMMANDS

Using cURL:

```
curl -X POST "https://study-bot-aliasyeda-duva.onrender.com/api/chat"
-H "Content-Type: application/json"
-d '{"user_id":"student","message":"What is artificial intelligence?"}'
```

Using Python:

```
import requests

response = requests.post(
"https://study-bot-aliasyeda-duva.onrender.com/api/chat",
json={"user_id": "test", "message": "Explain neural networks"}
)
print(response.json()["response"])
```

Sample Questions to Test:

- "What is the theory of relativity?"
- "Explain how photosynthesis works"
- "What are the three laws of motion?"
- "Tell me about the Roman Empire"
- "How do I solve quadratic equations?"

7. CONCLUSION

The Study Bot project successfully demonstrates:

- Integration of LLM APIs (Groq) for intelligent responses
- Persistent memory using MongoDB for context retention
- RESTful API design with FastAPI
- Cloud deployment on Render
- Handling of real-world deployment challenges

The bot effectively answers study-related questions while maintaining conversation context across sessions, fulfilling all project requirements.